Density Decomposition on Hypergraphs

1st Xiaoyu Leng
Department of Computer Science
Beijing Institute of Technology
Beijing , China
lengxiaoyu@bit.edu.cn

2nd Hongchao Qin
Department of Computer Science
Beijing Institute of Technology
Beijing , China
hcqin@bit.edu.cn

3rd Rong-Hua Li
Department of Computer Science
Beijing Institute of Technology
Beijing , China
rhli@bit.edu.cn

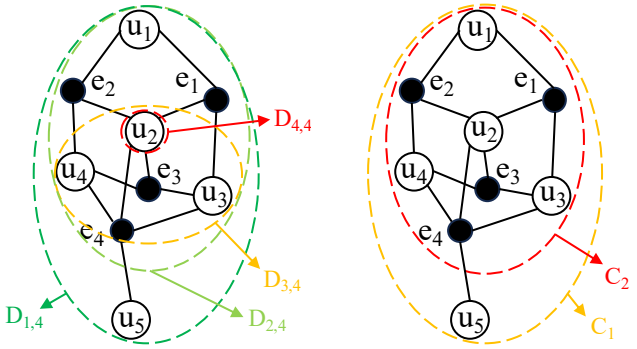
***Abstract*—Hypergraphs are widely used to model higher-order relations in which one interaction may involve multiple entities. Dense subhypergraph decomposition is a fundamental tool for organizing such data into hierarchical cohesive regions. Existing methods, such as k-core and nbr-k-core, mainly rely on degree-style constraints and thus provide only surrogate measures of density, often missing fine-grained density variations induced by multi-way interactions. We propose a density-oriented decomposition framework based on a new (k, δ)-dense subhypergraph model. Here, k represents an integer density level, while δ bounds the contribution of each hyperedge to density. By introducing an egalitarian δ-orientation, our model fairly allocates bounded hyperedge contributions among vertices and defines dense subhypergraphs through both allocated density and contribution-preserving reachability. We prove that the resulting dense subhypergraphs are unique and form a nested hierarchy. To compute the decomposition efficiently, we develop a fair-stable mining algorithm with time complexity $O(nm\delta)$, improving over the path-based bound $O(m^2\delta^2)$. Building on this result, we design a divide-and-conquer decomposition framework that reduces the full decomposition cost from $O(nm\delta \cdot d^E_{\max} \cdot k_{\max})$ to $O(nm\delta \cdot d^E_{\max} \cdot \log k_{\max})$. Experiments on nine real-world hypergraph datasets show that our method produces finer-grained and less redundant decomposition hierarchies than existing baselines, with up to a $20\times$ increase in the number of layers and up to a $10\times$ improvement in efficiency. Case studies further demonstrate that our decomposition uncovers compact and interpretable structures overlooked by degree-based methods.**

***Index Terms*—hypergraph, density, density decomposition**

I. Introduction

Hypergraphs provide a natural abstraction for modeling higher-order relationships in which one interaction may involve more than two entities. Such structures arise in many data management and mining scenarios, including multi-party transactions, group interactions in social platforms, collaborative activities in recommendation systems, and higher-order relations in scientific domains. Compared with ordinary graphs, hypergraphs preserve the cardinality and semantics of group relations, enabling more faithful analysis of cohesive structures formed by multi-way interactions [1], [2]. A fundamental task in hypergraph analysis is dense subhypergraph decomposition, which organizes vertices into nested cohesive regions for community detection, pattern discovery, anomaly detection, and structural summarization [3], [4], [5], [6].

Existing hypergraph decomposition models are mainly based on local degree-style constraints. The representative k-core model requires every vertex to participate in at least k hyperedges [4], while nbr-k-core further considers the number of neighboring vertices induced by incident hyperedges [5], [7]. These models are simple and scalable, but their criteria are still threshold surrogates of density rather than density itself. In a hypergraph, a large hyperedge may connect many vertices while contributing weakly to local cohesion. Counting incident hyperedges or neighboring vertices cannot reliably distinguish genuinely dense regions from broad but loosely connected groups. Consequently, degree-based decompositions may produce coarse, discontinuous, or redundant layers, and may merge structurally different regions into the same level. Figure 1 illustrates this limitation. The k-core decomposition in Figure 1(b) divides the hypergraph into only two layers, where vertices in C_1 and C_2 satisfy degree thresholds 1 and 2, respectively. However, this decomposition does not reveal the finer density variation inside the hypergraph. In contrast, the density-oriented decomposition in Figure 1(a) separates the same hypergraph into multiple density levels: $D_{4,4}$ corresponds to density level 4, $D_{3,4} \setminus D_{4,4}$ corresponds to density level 3, and $D_{1,4} \setminus D_{2,4}$ corresponds to density level 1. This example shows that meaningful density levels may be invisible to degree-based decomposition.



(a) $(k, 4)$-density decomposition (b) Core decomposition

Fig. 1. Density decomposition and core decomposition. Each e_i denotes a hyperedge, and each u_j denotes a vertex. An "edge" between e_i and u_j indicates that $u_j \in e_i$.

This issue is important in applications where the density level of a group carries semantic meaning. In legislative co-sponsorship hypergraphs, dense regions may correspond to stable coalitions repeatedly participating in related bills; in financial transaction hypergraphs, compact dense groups may

indicate suspicious coordination; and in biological interaction systems, cohesive modules are often formed by overlapping multi-way relations [1]. These applications require a decomposition that directly targets density, assigns each layer an interpretable density level, and remains computationally efficient for large hypergraphs. Designing such a decomposition is non-trivial. Hyperedges have variable cardinalities and may contribute to multiple vertices simultaneously. If each hyperedge contributes its full size, large hyperedges can dominate the density measure; if each hyperedge contributes only a bounded amount, the contribution must be allocated among its incident vertices in a principled and fair way. Therefore, density-oriented decomposition requires a controllable hyperedge contribution rule, a fair allocation mechanism, and an efficient algorithm for constructing the resulting hierarchy.

In this paper, we propose the (k, δ)-dense subhypergraph model, a new density decomposition framework for hypergraphs. For a parameter δ, each hyperedge contributes to at most δ vertices under a δ-orientation. We then use an egalitarian orientation to balance the allocated contributions among vertices. Based on this orientation, the (k, δ)-dense subhypergraph consists of vertices whose allocated density reaches level k, together with vertices structurally reachable from them through contribution-preserving hyperpaths. This formulation yields an integer-valued density hierarchy, where k directly represents the density level and δ controls the resolution of hyperedge contribution. We prove that the resulting (k, δ)-dense subhypergraphs are unique and form a nested hierarchy, making the decomposition well-defined.

Computing the proposed decomposition efficiently is challenging because a straightforward method needs to repeatedly search for and reverse reversible hyperpaths. To address this challenge, we develop a series of algorithms for both single dense subhypergraph mining and full density decomposition. Specifically, we first design a hyperpath-based method, a flow-based method, and a more scalable fair-stable method for mining a single (k, δ)-dense subhypergraph. We then build two decomposition algorithms on top of these mining procedures: a basic multi-layer peeling algorithm and an improved divide-and-conquer algorithm that exploits the nested structure of (k, δ)-dense subhypergraphs to reduce redundant computation. Finally, we validate the proposed model and algorithms on nine hypergraph datasets, showing that density-oriented decomposition produces finer-grained, less redundant, and more interpretable hierarchies than existing degree-based baselines.

To summarize, our main contributions are as follows.

- We propose the (k, δ)-dense subhypergraph model, which enables density-oriented hierarchical decomposition of hypergraphs. In this model, k represents the integer density level of each layer, while δ bounds the contribution of each hyperedge to the density. Unlike core-based and nbr-k-core decompositions [5], [7], our model directly characterizes density variations induced by multi-way relationships.
- We develop efficient algorithms for mining (k, δ)-dense subhypergraphs and computing the full density decomposition. We first propose a hyperpath-based method and a flow-based method that improves the time complexity from $O(m^2\delta^2)$ to $O(m^{1.5}\bar{d}_e^{1.5})$, where m is the number of hyperedges and $\bar{d}_e$ is the average hyperedge size. We further introduce a fair-stable method with time complexity $O(nm\delta)$, where n is the number of vertices. Based on this method, our divide-and-conquer decomposition algorithm reduces the overall cost from $O(nm\delta \cdot d^E_{\max} \cdot k_{\max})$ to $O(nm\delta \cdot d^E_{\max} \cdot \log k_{\max})$.
- We conduct extensive experiments on nine real-world hypergraph datasets. Compared with state-of-the-art nbr-k-core decomposition algorithms [5], [7], our approach achieves up to a $20\times$ increase in the number of decomposition layers and up to a $10\times$ improvement in computational efficiency. Case studies further show that our model uncovers compact and interpretable structures overlooked by degree-based methods in legislative and financial transaction hypergraphs.
- The code is available at https://github.com/xiaoyu-ll/IDH.

II. Problem Definition

Let $\mathcal{H} = (V, E)$ be an undirected and unweighted hypergraph, where V is the vertex set and E is the hyperedge set. Each hyperedge $e \in E$ is a subset of V, i.e., $e \subseteq V$ and $|e| \geq 1$. Let $n = |V|$ and $m = |E|$ denote the numbers of vertices and hyperedges, respectively. We use $d^E_{\max}$ and $d^E_{\min}$ to denote the maximum and minimum hyperedge sizes in $\mathcal{H}$, and define the average hyperedge size as $\bar{d}_e = \frac{1}{m}\sum_{e\in E}|e|$. For a vertex $u \in V$, its degree d_u is the number of hyperedges containing u. For a vertex set $S \subseteq V$, $E[S]$ denotes the set of hyperedges fully contained in S. We introduce a parameter $\delta \in [1, d^E_{\max}]$ to bound the contribution of each hyperedge in the density formulation.

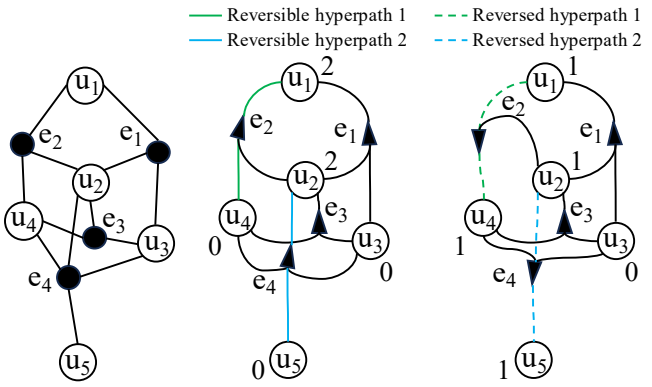
A. Directed Hypergraphs and Orientations

To allocate density contributions among vertices, we introduce an orientation mechanism for hyperedges. An orientation of $\mathcal{H}$ transforms each undirected hyperedge $e \in E$ into a directed hyperedge $\vec{e} = (T_e, H_e)$, where $T_e \subseteq e$ is the source set, $H_e \subseteq e$ is the target set, $T_e \cap H_e = \emptyset$, and $T_e \cup H_e = e$. The resulting directed hypergraph is denoted by $\vec{\mathcal{H}} = (V, \vec{E})$, where $\vec{E}$ is the set of directed hyperedges. For example, the directed hypergraphs in Figure 2(b)–(c) are orientations of the undirected hypergraph in Figure 2(a). In Figure 2(b), the directed hyperedge $\vec{e}_4 = (\{u_3, u_4, u_5\}, \{u_2\})$ has source vertices u_3, u_4, u_5 and target vertex u_2. Given a directed hypergraph $\vec{\mathcal{H}}$, the indegree of a vertex u is defined as $\vec{d}_u(\vec{\mathcal{H}}) = \left|\{\vec{e} = (T_e, H_e) \in \vec{E} \mid u \in H_e\}\right|$. For simplicity, we write $\vec{d}_u$ when the directed hypergraph.

Definition 1 (δ-Orientation): Given a hypergraph $\mathcal{H} = (V, E)$ and its orientation $\vec{\mathcal{H}} = (V, \vec{E})$, $\vec{\mathcal{H}}$ is called a δ-orientation of $\mathcal{H}$ if for every directed hyperedge $\vec{e} = (T_e, H_e) \in \vec{E}$ corresponding to an undirected hyperedge $e \in E$, we have $|H_e| = \min\{\delta, |e|\}$, where $\delta \in [1, d^E_{\max}]$.

B. Hyperpaths and Egalitarian Orientation

We next define hyperpaths and reversible hyperpaths, which formalize how density contributions can be transferred and balanced under a δ-orientation.



(a) initial hypergraph (b) 1-orientation (c) egalitarian 1-orientation

Fig. 2. Arbitrary and egalitarian 1-orientation.

Definition 2 (Hyperpath, Reversible Hyperpath): In a directed hypergraph $\vec{\mathcal{H}} = (V, \vec{E})$, a hyperpath from vertex s to t is a sequence: $s = u_0, \vec{e}_0, u_1, \ldots, \vec{e}_{l-1}, u_l = t$, where each $\vec{e}_i = (T_{e_i}, H_{e_i})$ satisfies $u_i \in T_{e_i}$ and $u_{i+1} \in H_{e_i}$. The hyperpath is *reversible* if $\vec{d}_t - \vec{d}_s \geq 2$.

If there exists a hyperpath from s to t, we say that s can reach t, denoted by $s \rightsquigarrow t$. Reversing a hyperpath means reversing the direction of every directed hyperedge along the path. For a reversible hyperpath $s \rightsquigarrow t$, such a reversal increases $\vec{d}_s$ by 1 and decreases $\vec{d}_t$ by 1, while leaving the indegrees of all other vertices unchanged. Thus, the indegree gap between s and t is reduced by 2. An egalitarian δ-orientation is an orientation in which no such balancing operation is possible, i.e., no reversible hyperpath exists.

Definition 3 (Egalitarian δ-Orientation): A δ-orientation $\vec{\mathcal{H}}$ is said to be an *egalitarian δ-orientation* if there exists no reversible hyperpath in $\vec{\mathcal{H}}$.

Example 1: Figure 2(b) shows an arbitrary 1-orientation. There exist two hyperpaths, $u_4 \rightsquigarrow u_1$ through $(u_4, \vec{e}_2, u_1)$ and $u_5 \rightsquigarrow u_2$ through $(u_5, \vec{e}_4, u_2)$. Since $\vec{d}_{u_1} - \vec{d}_{u_4} = 2$ and $\vec{d}_{u_2} - \vec{d}_{u_5} = 2$, both hyperpaths are reversible. Reversing them gives the orientation in Figure 2(c), where the corresponding indegree gaps are reduced by 2. As no reversible hyperpath remains, the resulting orientation is an egalitarian 1-orientation.

C. (k, δ)-Dense Subhypergraph

We now use egalitarian δ-orientations to define density levels and identify the corresponding dense subhypergraphs.

Definition 4 ((k,δ)-Dense Subhypergraph ($D_{k,\delta}$)): Given an undirected and unweighted hypergraph $\mathcal{H} = (V, E)$ and two non-negative integers k and δ, let $\vec{\mathcal{H}}$ be an arbitrary egalitarian δ-orientation of $\mathcal{H}$. Define the vertex set $S = \{u \in V \mid \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$. The (k, δ)-dense subhypergraph $D_{k,\delta}$ is the subhypergraph induced by $S \cup \{v \mid v \text{ can reach a vertex in } S\}$.

Example 2: Consider the egalitarian 1-orientation in Figure 2(c). Let $k = 1$. Then $S = \{u_1, u_2, u_4, u_5\}$. Since vertex u_3 can reach u_2 via the hyperpath $u_3 \rightsquigarrow u_2$ $(u_3, \vec{e}_3, u_2)$, the $(1, 1)$-dense subhypergraph is induced by $\{u_1, u_2, u_3, u_4, u_5\}$.

By Definition 4, we can derive the following properties.

Lemma 1: Given an egalitarian δ-orientation $\vec{\mathcal{H}}$ and its corresponding (k, δ)-dense subhypergraph $D_{k,\delta}$:

1) All hyperedges crossing from $D_{k,\delta}$ to $V \setminus D_{k,\delta}$ are oriented outward.
2) For any $u \in D_{k,\delta}$, we have $\vec{d}_u \geq k - 1$.
3) For any $u \notin D_{k,\delta}$, we have $\vec{d}_u \leq k - 1$.

The following theorem establishes that $D_{k,\delta}$ is cohesive internally and well-separated externally.

Theorem 1: Let $D_{k,\delta}$ be a (k, δ)-dense subhypergraph. Then:

- For any non-empty $X \subseteq D_{k,\delta}$, $\sum_{x \in X} \vec{d}_x > (k-1)|X|$.
- For any $Y \subseteq V \setminus D_{k,\delta}$, $\sum_{y \in Y} \vec{d}_y \leq (k-1)|Y|$.

Proof. Let $D = D_{k,\delta}$ and let $S = \{u \in V \mid \vec{d}_u \geq k\}$ be the seed set used in Definition 4 under a fixed egalitarian δ-orientation. We first prove the boundary property used below. Suppose that a directed hyperedge $\vec{e} = (T, H)$ has a target vertex in D and a source vertex $z \notin D$. Then z reaches this target in one step and hence reaches a vertex of S whenever the target is in S, or reaches a vertex that itself reaches S otherwise. This implies $z \in D$, a contradiction. Therefore every hyperedge crossing the cut $(D, V \setminus D)$ is oriented outward from D, i.e., no outside vertex contributes indegree to a vertex of D across the cut. For the first claim, take any non-empty $X \subseteq D$. If $X \cap S \neq \emptyset$, then vertices in S have indegree at least k, while Lemma 1 gives $\vec{d}_x \geq k - 1$ for all remaining vertices of D. Hence $\sum_{x \in X} \vec{d}_x > (k-1)|X|$. It remains to consider $X \cap S = \emptyset$. Since every vertex in $D \setminus S$ reaches S, if all vertices of X had indegree exactly $k - 1$ and no hyperedge left X along a reachable direction, then no vertex of X could reach S. Thus along some hyperpath from X to S there is a first step leaving X through a hyperedge whose target lies outside X. The predecessor vertex in X is a source of this directed hyperedge and, by egalitarianity, cannot have indegree below $k-1$; otherwise a reversible transfer from the endpoint with indegree at least k back toward it would exist along the suffix of the path. Consequently at least one vertex in X has indegree at least k, and the strict lower bound follows. For the second claim, Lemma 1 gives $\vec{d}_y \leq k - 1$ for every $y \notin D$. Summing this pointwise bound over $Y \subseteq V \setminus D$ yields $\sum_{y \in Y} \vec{d}_y \leq (k-1)|Y|$.

Remark on δ. The parameter δ controls how much contribution a hyperedge can allocate to its incident vertices. A larger δ allows each hyperedge to support more vertices, leading to a finer density scale, whereas a smaller δ imposes a stricter contribution cap and yields a coarser decomposition. Thus, δ provides a natural knob for multi-resolution hypergraph analysis, as empirically studied in Exp-9.

D. Properties of the Decomposition

We now analyze the theoretical properties of the (k, δ)-dense subhypergraph model and its induced decomposition.

Uniqueness of decomposition. The definition of $D_{k,\delta}$ relies solely on the existence of an egalitarian δ-orientation. Therefore, regardless of which specific egalitarian orientation is adopted, the resulting $D_{k,\delta}$ remains exactly the same.

Theorem 2: Given a hypergraph $\mathcal{H}$ and parameters k and δ, the (k,δ)-dense subhypergraph $D_{k,\delta}$ is unique.

Proof. Fix k and δ. Let $\vec{\mathcal{H}}_1$ and $\vec{\mathcal{H}}_2$ be two egalitarian δ-orientations, and let D_1 and D_2 be the vertex sets returned by Definition 4 from these two orientations. We show that $D_1 = D_2$. Assume, for contradiction, that $D_1 \setminus D_2$ is non-empty and denote this difference by X. Applying Theorem 1 to D_1 under $\vec{\mathcal{H}}_1$, every non-empty subset of D_1 has aggregate indegree larger than $(k-1)$ times its size; in particular, $\sum_{x\in X} \vec{d}_x^{(1)} > (k-1)|X|$. Applying the outside part of Theorem 1 to D_2 under $\vec{\mathcal{H}}_2$ gives $\sum_{x\in X} \vec{d}_x^{(2)} \le (k-1)|X|$. It remains to note that egalitarian δ-orientations have the same canonical high-level closure: if a set violates the outside upper bound in one egalitarian orientation, then a sequence of reversible hyperpath reversals from any other δ-orientation must also place the set inside the closure of the vertices with indegree at least k. This is because each reversal only transfers one unit of indegree along an existing reachable hyperpath and cannot change the cut condition that separates a closed set from its complement. Hence the two displayed inequalities cannot hold for the same closed difference set X. The symmetric argument rules out $D_2 \setminus D_1 \neq \emptyset$. Therefore $D_1 = D_2$, and the (k,δ)-dense subhypergraph is unique.

Hierarchy of (k,δ)-dense subhypergraphs. The (k,δ)-dense subhypergraphs form a natural nested hierarchy, with higher k values corresponding to smaller and denser regions.

Theorem 3: For any $k^+ \ge k$, we have $D_{k^+,\delta} \subseteq D_{k,\delta}$.

Proof. Assume for contradiction that $X = D_{k^+,\delta} \setminus D_{k,\delta}$ is non-empty. Since $X \subseteq D_{k^+,\delta}$, Theorem 1 applied to level k^+ yields $\sum_{x\in X} \vec{d}_x > (k^+-1)|X|$. Because $k^+ \ge k$, this implies $\sum_{x\in X} \vec{d}_x > (k-1)|X|$. On the other hand, $X \subseteq V \setminus D_{k,\delta}$, so the outside part of Theorem 1 applied to level k gives $\sum_{x\in X} \vec{d}_x \le (k-1)|X|$. The two inequalities contradict each other. Hence $D_{k^+,\delta} \subseteq D_{k,\delta}$.

Density metrics and interpretability. We next analyze the (k,δ)-dense subhypergraph using indegree-based density.

Definition 5 (Indegree-Based Density): Given a directed hypergraph $\vec{\mathcal{H}} = (V, \vec{E})$ and a vertex set X, its indegree-based density is defined as $\rho_d(X) = \sum_{x\in X} \vec{d}_x / |X|$.

Lemma 2: For any $X \subseteq D_{k,\delta}$ and $Y \subseteq V \setminus D_{k,\delta}$, we have $\rho_d(X) > k-1 \ge \rho_d(Y)$.

Proof. For any non-empty $X \subseteq D_{k,\delta}$, Theorem 1 gives $\sum_{x\in X} \vec{d}_x > (k-1)|X|$. Dividing by $|X|$ yields $\rho_d(X) > k-1$. Similarly, for any non-empty $Y \subseteq V \setminus D_{k,\delta}$, Theorem 1 gives $\sum_{y\in Y} \vec{d}_y \le (k-1)|Y|$, and hence $\rho_d(Y) \le k-1$. The empty-set case is vacuous and is ignored in the density ratio.

Definition 6 (Layer Density): For any non-negative integer k, the density of the (k,δ)-layer is defined as $\rho_d(L_{k,\delta}) = \rho_d(D_{k+1,\delta}, D_{k,\delta}) = \sum_{x\in D_{k,\delta}\setminus D_{k+1,\delta}} \vec{d}_x / |D_{k,\delta} \setminus D_{k+1,\delta}|$.

Lemma 3: For any non-negative integer k, we have $(k-1) < \rho_d(D_{k+1,\delta}, D_{k,\delta}) \le k$. Thus, $\lceil \rho_d(L_{k,\delta}) \rceil = \lceil \rho_d(D_{k+1,\delta}, D_{k,\delta}) \rceil = k$, indicating that layer $L_{k,\delta}$ corresponds to density level k.

Proof. Let $L_{k,\delta} = D_{k,\delta} \setminus D_{k+1,\delta}$ and assume that this layer is non-empty. Since $L_{k,\delta} \subseteq D_{k,\delta}$, Theorem 1 applied to $D_{k,\delta}$ gives $\sum_{x\in L_{k,\delta}} \vec{d}_x > (k-1)|L_{k,\delta}|$. Therefore $\rho_d(L_{k,\delta}) > k-1$. Moreover, $L_{k,\delta} \subseteq V \setminus D_{k+1,\delta}$. Applying the outside part of Theorem 1 to $D_{k+1,\delta}$ yields $\sum_{x\in L_{k,\delta}} \vec{d}_x \le k|L_{k,\delta}|$, and hence $\rho_d(L_{k,\delta}) \le k$. Combining the two bounds gives $(k-1) < \rho_d(L_{k,\delta}) \le k$, so $\lceil \rho_d(L_{k,\delta}) \rceil = k$.

Definition 7 (Integral Dense Number (IDN)): For $u \in D_{k,\delta} \setminus D_{k+1,\delta}$, its *Integral Dense Number (IDN)* is defined as $\bar{r}_u^{\delta} = k$.

The above results establish that the proposed model induces a well-defined density hierarchy. For a fixed δ, the subhypergraphs are nested as k increases, and each non-empty layer $D_{k,\delta} \setminus D_{k+1,\delta}$ corresponds to the integer density level k. Thus, the density decomposition problem reduces to computing all distinct non-empty (k,δ)-dense subhypergraphs efficiently.

E. Density and Conductance Guarantee

We now connect our orientation-based density with standard undirected hypergraph density and conductance.

Definition 8 (degree-Density): Given a hypergraph $\mathcal{H}$ and a subhypergraph X, the density of X is defined as $\rho(X) = \frac{\sum_{x\in X} d_x(X)}{|X|}$.

The subhypergraph $X \subseteq V$ that maximizes the density $\rho(X)$ is recognized as the *densest subhypergraph* of $\mathcal{H}$.

Lemma 4: For any $D_{k,\delta} \neq \emptyset$, we have $\rho(D_{k,\delta}) \ge \rho_d(D_{k,\delta}) > k-1$.

Proof. Let $X = D_{k,\delta}$. By Lemma 1, every hyperedge crossing from X to $V \setminus X$ is oriented outward, so no crossing hyperedge contributes to the indegree of a vertex in X. Therefore every unit counted by $\sum_{x\in X} \vec{d}_x$ is contributed by an internal hyperedge of X. Since $d_x(X)$ counts all internal hyperedges incident to x, every oriented contribution to x is also counted in $d_x(X)$. Hence $\sum_{x\in X} \vec{d}_x \le \sum_{x\in X} d_x(X)$, and dividing by $|X|$ gives $\rho(X) \ge \rho_d(X)$. The strict lower bound $\rho_d(D_{k,\delta}) > k-1$ follows from Theorem 1 with $X = D_{k,\delta}$.

Definition 9 (Internalization Coefficient): For a subhypergraph X, define the internalization coefficient as $\theta(X) = \frac{\sum_{e\in E(X)} |H(e)|}{\sum_{x\in X} \vec{d}_x} \in [0,1]$, where $\theta(X) = 0$ if $\sum_{x\in X} \vec{d}_x = 0$. Let $f_k = |S_k|/|D_{k,\delta}|$ denote the fraction of vertices in S_k within $D_{k,\delta}$.

Lemma 5: For any $D_{k,\delta} \neq \emptyset$, $\rho_d(D_{k,\delta}) \ge (k-1) + f_k$.

Proof. By Definition 4, each vertex $s \in S_k$ has indegree at least k, and each vertex $x \in D_{k,\delta} \setminus S_k$ has indegree at least $k-1$ but can reach some s. Thus $\rho_d(D_{k,\delta}) \ge f_k \cdot k + (1-f_k)(k-1) = (k-1) + f_k$.

Lemma 6: For any $X \subseteq V$, $\rho(X) \ge \theta(X)\,\rho_d(X)$.

Proof. Since $\sum_{x\in X} d_x(X) = \sum_{e\in E(X)} |e| \ge \sum_{e\in E(X)} |H(e)| = \theta(X) \sum_{x\in X} \vec{d}_x$, dividing both sides by $|X|$ yields the inequality.

Theorem 4 (Density Guarantee): For any parameters k and δ, we have $\rho(D_{k,\delta}) \ge \theta(D_{k,\delta})\big((k-1) + f_k\big)$.

Proof. Vertices in S_k satisfy $\vec{d} \ge k$, while those in $D_{k,\delta} \setminus S_k$ have $\vec{d} \ge k-1$ and can reach some $s \in S_k$. Hence, $\rho_d(D_{k,\delta}) \ge (k-1) + f_k$. Moreover, by Lemma 6, $\rho(X) \ge \theta(X)\,\rho_d(X)$. Combining the two gives the desired

bound. Equivalently, in the edge–vertex view, $\frac{|E(D_{k,\delta})|}{|D_{k,\delta}|} = \frac{|E(D_{k,\delta})|\cdot\bar{r}(D_{k,\delta})}{|D_{k,\delta}|\cdot\bar{r}(D_{k,\delta}))} = \frac{\rho(D_{k,\delta})}{\bar{r}(D_{k,\delta})} \geq \frac{\theta((k-1)+f_k)}{\bar{r}(D_{k,\delta})}$.

Theorem 5 (Conductance Bound): For any $X \subseteq V$, the conductance satisfies$\phi(X) \leq 1 - \frac{\rho(X)}{\bar{d}(X)}$, where $\bar{d}(X) = \mathrm{vol}(X)/|X|$. In particular, $\phi(D_{k,\delta}) \leq 1 - \frac{\theta(D_{k,\delta})((k-1)+f_k)}{\bar{d}(D_{k,\delta})}$.

Proof. Let $\mathrm{vol}(X) = \sum_{x\in X} d_x$ and let $\partial(X)$ denote the number of incidences contributed by hyperedges that cross the cut between X and $V \setminus X$. Every incidence of a vertex in X is either internal to X or belongs to a crossing hyperedge. Therefore $\partial(X) \leq \mathrm{vol}(X) - \sum_{x\in X} d_x(X)$. By Definition 8, $\sum_{x\in X} d_x(X) = \rho(X)|X|$, and by definition $\mathrm{vol}(X) = \bar{d}(X)|X|$. Dividing by $\mathrm{vol}(X)$ gives $\phi(X) \leq 1 - \rho(X)/\bar{d}(X)$. For $X = D_{k,\delta}$, Theorem 4 gives $\rho(D_{k,\delta}) \geq \theta(D_{k,\delta})((k-1)+f_k)$. Substituting this lower bound into the preceding inequality proves the stated bound.

F. Problem Statements and Challenges

Problem 1: Dense Subhypergraph Mining (DSM). Given a hypergraph $\mathcal{H} = (V, E)$ and two integers k and δ, compute the (k, δ)-dense subhypergraph $D_{k,\delta}$.

Problem 2: Density Subhypergraph Decomposition (DSD). Given a hypergraph $\mathcal{H} = (V, E)$, compute all non-empty (k, δ)-dense subhypergraphs over different values of k and δ.

A naive solution to DSM is to search over δ-orientations until an egalitarian one is obtained. Since each hyperedge can allocate its contribution to up to δ target vertices, the orientation space may grow exponentially, with an intuitive worst-case size up to $O(2^{m\delta})$. Although reversible hyperpath operations can gradually move an orientation toward egalitarianity, applying them naively may still require exploring a large number of states. Thus, the first challenge is to compute $D_{k,\delta}$ without explicitly searching this exponential space.

DSD is even more demanding: independently solving DSM for every (k, δ) pair would repeat substantial computation across nested dense subhypergraphs. The second challenge is therefore to exploit the hierarchy of (k, δ)-dense subhypergraphs to construct the full decomposition efficiently.

The above discussion leads to two algorithmic challenges:

- How can we compute a (k, δ)-dense subhypergraph without searching over exponentially many orientations?
- How can we reuse the nested structure of (k, δ)-dense subhypergraphs to accelerate full density decomposition?

III. DENSE SUBHYPERGRAPH MINING

This section presents four algorithms for computing the (k, δ)-dense subhypergraph. A naive approach that enumerates reversible hyperpaths may incur exponential time, with a worst-case complexity of $O(2^{m\delta})$. To avoid exhaustive enumeration, we exploit structural properties of hyperpaths and reduce the indegrees of high-indegree vertices by reversing suitable hyperpaths. We first introduce DSM-PATH, which repeatedly locates and reverses one reversible hyperpath using BFS and runs in $O(m^2\delta^2)$ time. We then develop DSM-FLOW, which formulates the reversal process as a max-flow problem and eliminates multiple reversible hyperpaths simultaneously, reducing the time complexity to $O(m^{1.5}\bar{d}_e^{1.5})$. Building on this formulation, DSM-FLOW+ improves the initial orientation while preserving the same asymptotic complexity. Finally, DSM-ALL avoids the memory overhead of flow-based methods and achieves $O(nm\delta)$ time by eliminating all relevant reversible hyperpaths.

Algorithm 1: DSM-PATH$(\mathcal{H}, k, \delta)$

Input: A hypergraph $\mathcal{H}$; two non-negative integers k, δ
Output: (k, δ)-dense subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

1 Arbitrarily obtain a δ-orientation $\vec{\mathcal{H}}$ of $\mathcal{H}$;
2 **while** *True* **do**
3 **if** $\exists$ *a reversible hyperpath* $s \rightsquigarrow t$ **then**
4 reverse the hyperpath $s \rightsquigarrow t$;
5 **else** break;
6 $S \leftarrow \{u \in V | \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$;
7 $D_{k,\delta} \leftarrow S \cup \{v | v$ can reach a vertex in $S\}$;
8 **return** $D_{k,\delta}$;

A. The BFS-based Algorithm: DSM-PATH

The algorithm performs BFS to identify and reverse a reversible hyperpath. The DSM-PATH algorithm is shown in Algorithm 1. First, it obtains an arbitrary δ-orientation of $\mathcal{H}$ (Line 1), which serves as the initial directed hypergraph. Then, in the while loop (Lines 2–5), the algorithm finds (Line 3) and reverses (Line 4) a reversible hyperpath via BFS, thereby reducing the indegree imbalance between vertices and progressively improving the orientation. Each iteration invokes one BFS and removes one reversible hyperpath. The loop terminates when no reversible hyperpath can be found (Line 5). Finally, the algorithm obtains the (k, δ)-dense subhypergraph $D_{k,\delta}$ according to Definition 4 (Line 7).

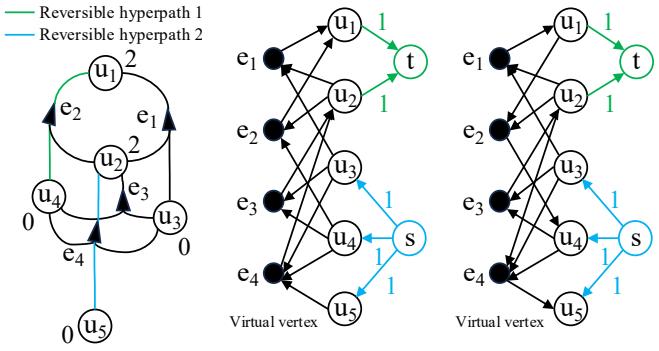
According to Definition 4, the DSM-PATH algorithm correctly outputs the (k, δ)-dense subhypergraph $D_{k,\delta}$.

Theorem 6 (Complexity of Algorithm DSM-PATH): The time and space complexity are $O(m^2\delta^2)$ and $O(n + m)$.

Proof. Let $\overline{S} = \{u \in V \mid \vec{d}\, u(\vec{\mathcal{H}}) \geq k\}$ be the initial set of high-indegree vertices in the δ-orientation obtained in Line 1 of Algorithm 1. Reversing an $s \rightsquigarrow t$ hyperpath does not create new high-indegree vertices and decreases the indegree of some vertex in $\overline{S}$ by one. Thus, the number of reversals is at most $\sum x \in \overline{S}\vec{d}_x \leq m\delta$. Since each hyperpath search and reversal takes $O(m\delta)$ time, the total time complexity is $O(m^2\delta^2)$, and the space complexity is $O(n + m)$.

B. The Flow-based Algorithm: DSM-FLOW

To overcome the inefficiency of DSM-PATH, which may reverse up to $O(m\delta)$ hyperpaths, we introduce the DSM-FLOW algorithm to efficiently separate (k, δ)-dense vertices from others. Leveraging the strength of network flow in vertex separation, DSM-FLOW eliminates all relevant reversible $s \rightsquigarrow t$ hyperpaths in one pass via a flow network. The core idea builds on augmenting hyperpath algorithms, adapting max-flow techniques to remove reversible hyperpaths. Inspired by



(a) 1-orientation (b) re-orientation network (c) egalitarian orientation

Fig. 3. An example of the re-orientation network.

the reorientation network of Bezakova et al. [8], we design a novel reorientation hypergraph network.

Definition 10 (re-orientation hypergraph network): Given a δ-orientation $\overrightarrow{\mathcal{H}} = (V, \overrightarrow{E})$ and an integer d, the re-orientation hypergraph network is constructed as a weighted factor graph, in which each hyperedge is treated as a vertex in the network, augmented with an additional source vertex s and sink vertex t. The weight assigned to each arc between two vertices represents the capacity of the arc. Specifically, the re-orientation network with parameter d is defined as $(V \cup V_E \cup s, t, A, c)$, where

1) $\langle u, v_e\rangle \in A, c(u, v_e) = 1$, if $u \in T, v_e \in V_E, \vec{e} = \{T, H\}$;
2) $\langle v_e, u\rangle \in A, c(v_e, u) = 1$, if $v_e \in V_E, u \in H, \vec{e} = \{T, H\}$;
3) $\langle s, u\rangle \in A, c(s, u) = \vec{d} - \vec{d}_u(\overrightarrow{H})$, if $\vec{d}_u(\overrightarrow{H}) < d$;
4) $\langle u, t\rangle \in A, c(u, t) = \vec{d}_u(\overrightarrow{H})$ - d, if $\vec{d}_u(\overrightarrow{H}) > d$.

By Definition 10, the *re-orientation hypergraph network* uses a parameter d to separate vertices by indegrees. To obtain $D_{k,\delta}$, we set $d = k - 1$. Consequently, the source s connects to vertices with an indegree less than $k - 1$, while the sink t links to vertices with an indegree greater than $k - 1$. Upon completion of the maximum flow algorithm, no augmentation paths remain in the residual network, indicating no reversible hyperpaths from s to t. Hence, all reversible hyperpaths are reversed and $D_{k,\delta}$ can be obtained.

Example 3: Given the 1-orientation in Figure 2(b) (Figure 3(a)) and k = 2, the corresponding re-orientation hypergraph network is shown in Figure 3(b), where the capacity of each arc is 1. For vertices in V, $k-1$ is the pivot of indegree. Source s is connected to the vertices whose indegree does not reach the pivot, thus it is connected to u_3, u_4, u_5 with a capacity of $k - 1 - \vec{d}_{u_3}/\vec{d}_{u_4}/\vec{d}_{u_5}$=1. Sink t is connected to the vertices whose indegree exceeds the pivot, thus it is connected to u_1 and u_2 with a capacity of $\vec{d}_{u_1}/\vec{d}_{u_2}$-$(k-1)$=1. After computing maximum flow, the network is Figure 3(c).

We design a network–flow–based algorithm, DSM-FLOW, as outlined in Algorithm 2. The algorithm first generates an arbitrary initial orientation of the hypergraph (Line 1), and then constructs the re-orientation network (Lines 4–10). A

Algorithm 2: DSM-FLOW$(\mathcal{H}, k, \delta)$

Input: a hypergraph $\mathcal{H}$, two non-negative integer k, δ.
Output: (k, δ)-*dense* subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

1 Arbitrarily obtain an δ-orientation $\overrightarrow{\mathcal{H}}$ of $\mathcal{H}$;
2 $V' \leftarrow V \cup \{s, t\} \cup V_E$, $d \leftarrow k - 1$;
3 **for** *each* $\langle u, v_e\rangle, u \in T, v_e \in V_E, \vec{e} = \{T, H\} \in \overrightarrow{E}$ **do**
4 add arc$\langle u, v_e\rangle$ to A and let $c(u, v_e) \leftarrow 1$;
5 **for** *each* $\langle v_e, u\rangle, v \in V_e, u \in H, \vec{e} = \{T, H\} \in \overrightarrow{E}$ **do**
6 add arc$\langle v_e, u\rangle$ to A and let $c(v_e, u) \leftarrow 1$;
7 **for** *each* $u, \vec{d}_u(\overrightarrow{H}) < d$ **do**
8 add arc$\langle s, u\rangle$ to A and let $c(s, u) \leftarrow d - \vec{d}_u(\overrightarrow{H})$;
9 **for** *each* $u, \vec{d}_u(\overrightarrow{H}) > d$ **do**
10 add arc$\langle u, t\rangle$ to A and let $c(u, t) \leftarrow \vec{d}_u(\overrightarrow{H})$ - d;
11 Compute the maximum flow value f_{max} of (V', A, c);
// Copy the residual network to $\overrightarrow{\mathcal{H}}$
12 **for** *each* $(u_x, v_e, u_y), u_x, u_y \in V, v_e \in V_E$ **do**
13 **if** $\langle u_x, v_e\rangle \in A, \langle v_e, u_y\rangle \in A$ *are saturated* **then**
14 reverse $\langle u_x, v_e\rangle$,$\langle v_e, u_y\rangle$;// $u_x \in H, u_y \in T$
15 Same as lines 6-7 in Algorithm 1;
16 **return** $D_{k,\delta}$;

maximum flow is subsequently computed on this network (Line 11), after which all reversible hyperpaths are reversed according to the resulting flow (Lines 12–14). Finally, the vertices remaining in the resulting oriented hypergraph form the (k, δ)-dense subhypergraph $D_{k,\delta}$, which is returned as the output (Line 15). We next analyze the correctness and computational complexity of DSM-FLOW.

Theorem 7 (Correctness of Algorithm DSM-FLOW *):* Algorithm DSM-FLOW correctly outputs $D_{k,\delta}$.

Proof. Set $d = k - 1$. In the re-orientation network, a path from the source s to the sink t has the form $s, u_0, v_{e_0}, u_1, \ldots, v_{e_{\ell-1}}, u_\ell, t$. By construction, s is adjacent only to vertices with indegree below d, t is adjacent only from vertices with indegree above d, and every middle segment u_i, v_{e_i}, u_{i+1} corresponds exactly to a directed hyperedge in which u_i is a source and u_{i+1} is a target. Hence each augmenting path in the network corresponds to a reversible hyperpath from a low-indegree vertex to a high-indegree vertex in the oriented hypergraph. Sending one unit of flow along this path is equivalent to reversing the corresponding hyperpath, which increases the first vertex indegree by one and decreases the last vertex indegree by one while preserving the δ-orientation constraint on every hyperedge. After a maximum flow is computed, the residual network contains no augmenting s–t path. Therefore no remaining reversible hyperpath can transfer one unit of indegree from a vertex with indegree larger than $k - 1$ to a vertex with indegree smaller than $k - 1$. The resulting orientation is stable with respect to the threshold $k - 1$. Let $S = \{u \mid \vec{d}_u \geq k\}$. Lines for extracting the output return exactly the closure of S under reachability in this stable orientation, which is precisely Definition 4. By Theorem 2, this set is the unique $D_{k,\delta}$, independent of the initial orientation.

Algorithm 3: DSM-FLOW+$(\mathcal{H}, k, \delta)$

Input: a hypergraph $\mathcal{H}$, two non-negative integer k, δ.
Output: (k, δ)-*dense* subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

1 $\vec{E} \leftarrow \emptyset$, $\vec{\mathcal{H}} \leftarrow (V, \vec{E})$;
2 **for** *each* $e \in E$ **do**
3 $\quad V_e \leftarrow$ select δ vertices with lowest degree in e;
4 $\quad \vec{e} = \{\{e \setminus V_e\}, \{V_e\}\}$, $\vec{E} \leftarrow \vec{E} \cup \vec{e}$;
5 Same as lines 2-17 in Algorithm 2;
6 **return** $D_{k,\delta}$;

Thus Algorithm 2 is correct.

Theorem 8 (Complexity of Algorithm DSM-FLOW*):* The time and space complexity are $O(m^{1.5}\bar{d}_e^{1.5})$ and $O(n + m\bar{d}_e)$.

Proof. The constructed reorientation network is an AUC-2 network, where all edges have unit capacity except those incident to the source or sink [9]. Maximum flow on such a network can be computed in $O(|E|^{1.5})$ time using $O(|E|)$ space. In our construction, each hyperedge contributes a number of network edges proportional to its size; hence the network contains $O(m\bar{d}_e)$ edges, together with $O(n)$ vertex-related nodes. Therefore, DSM-FLOW runs in $O(m^{1.5}\bar{d}_e^{1.5})$ time and requires $O(n + m\bar{d}_e)$ space.

C. The Improved Algorithm: DSM-FLOW+

We further propose DSM-FLOW+, an enhanced flow-based algorithm that improves the initial orientation used by DSM-FLOW. Instead of choosing the head of each hyperedge arbitrarily, DSM-FLOW+ orients each hyperedge toward a vertex with the smallest current indegree (Algorithm 3, Lines 2–4). This greedy initialization directly targets the maximum indegree and produces a more balanced orientation before the max-flow refinement. As a result, the subsequent flow computation starts from a better orientation.

Theorem 9 (Complexity of Algorithm DSM-FLOW+*):* The time and space complexity are $O(m^{1.5}\bar{d}_e^{1.5})$ and $O(n + m\bar{d}_e))$.

Proof. DSM-FLOW+ differs from DSM-FLOW only in its initial δ-orientation step. For each hyperedge, the δ vertices with the lowest current indegrees can be selected in linear time with respect to the hyperedge size, so the total initialization cost is $O(m\bar{d}_e)$. The remaining steps are identical to DSM-FLOW and take $O(m^{1.5}\bar{d}_e^{1.5})$ time. Thus, the overall time complexity remains $O(m^{1.5}\bar{d}_e^{1.5})$, and the space complexity is $O(n + m\bar{d}_e)$.

D. The Memory-Efficient Algorithm: DSM-ALL

Although DSM-FLOW accelerates the elimination of reversible hyperpaths through a flow-network formulation, constructing and storing the network can incur substantial memory overhead. To improve scalability, we propose DSM-ALL, which avoids explicitly building the flow network while preserving its key separation principle. Instead of computing a maximum flow, DSM-ALL directly identifies vertices that are reachable through reversible hyperpaths and eliminates all relevant reversible hyperpaths between high-indegree and low-indegree vertices. As a result, no vertex outside the selected high-indegree set can reach a vertex inside it via a reversible hyperpath, which satisfies the required dense subhypergraph condition while reducing memory usage.

Algorithm 4: DSM-ALL$(\mathcal{H}, k, \delta)$

Input: a hypergraph $\mathcal{H}$, two non-negative integer k, δ.
Output: (k, δ)-*dense* subhypergraph $D_{k,\delta}$ of $\mathcal{H}$

1 Arbitrarily obtain a δ-orientation $\vec{\mathcal{H}}$ of $\mathcal{H}$;
2 $\overline{S} \leftarrow \{u \in V | \vec{d}_u(\vec{\mathcal{H}}) \geq k\}$;
3 **for** each $u \in \overline{S}$ **do** REACHOUT(u, k);
4 **while** *True* **do**
5 $\quad$ **if** $\exists\ u \in \overline{S}$ with $\vec{d}_u < k$ **then** OUT(u, k);
6 $\quad$ **else break**;
7 $D_{k,\delta} \leftarrow \overline{S} \cup \{v | v$ can reach a vertex in $\overline{S}\}$;
8 **return** $D_{k,\delta}$;
9 **Function** REACHOUT(u, k)**:**
10 $\quad$ **if** $\exists$ *a reversible hyperpath* $s \rightsquigarrow u$ *with* $s \in V \setminus \overline{S}$ **then**
11 $\quad\quad$ reverse the hyperpath $s \rightsquigarrow u$;
12 $\quad\quad$ **if** $\vec{d}_s \geq k$ **then** $\overline{S} \leftarrow \overline{S} \cup s$;
13 **Function** REACHIN(u, k)**:**
14 $\quad$ **if** $\exists$ *a reversible hyperpath* $u \rightsquigarrow s$ *with* $s \in V \setminus \overline{S}$ **then**
15 $\quad\quad$ reverse the hyperpath $u \rightsquigarrow s$;
16 **Procedure** OUT(u, k)**:**
17 $\quad$ initindegree $\leftarrow \vec{d}_u$;
18 $\quad$ **if** $\exists$ *reversible* $s \rightsquigarrow u$*/*$u \rightsquigarrow s$, $s \in \overline{S} \cap N(u)$ **then**
19 $\quad\quad$ reverse the hyperpath;
20 $\quad$ **if** initindegree $> \vec{d}_u$ **then** REACHIN(u, k);
21 $\quad$ **else** initindegree $< \vec{d}_u$ **then** REACHOUT(u, k);
22 $\quad$ **if** $\vec{d}_u < k$ **then** $\overline{S} \leftarrow \overline{S} \setminus u$;

Algorithm 4 describes DSM-ALL. It first initializes $\bar{S}$ as the set of vertices whose indegree is at least k (Line 2). For each vertex outside $\bar{S}$, the algorithm searches for a reversible hyperpath to a vertex in $\bar{S}$ and reverses it if one exists (Lines 10–11). If this reversal increases the external vertex's indegree to k, the vertex is added to $\bar{S}$ (Line 12). This process eliminates cross-boundary reversible hyperpaths from $V \setminus \bar{S}$ to $\bar{S}$. During these reversals, some vertices in $\bar{S}$ may drop below the indegree threshold k and need to be removed (Line 5). Before removing such a vertex, DSM-ALL first reverses any remaining reversible hyperpaths between it and other vertices in $\bar{S}$ (Lines 18–19), and then checks reversible hyperpaths from this vertex to vertices outside $\bar{S}$ (Lines 20–21). If its indegree is still below k after these updates, the vertex is permanently removed from $\bar{S}$ (Line 22).

Theorem 10 (Correctness of Algorithm DSM-ALL*):* Algorithm DSM-ALL correctly outputs $D_{k,\delta}$

Proof. Let $\overline{S}$ denote the maintained candidate seed set. The algorithm preserves the following invariant after every call to REACHOUT, REACHIN, or OUT: every vertex in $\overline{S}$ is either currently of indegree at least k or is scheduled for repair/removal, and every performed reversal is along a reversible hyperpath, so the orientation remains a valid δ-orientation. Initially, $\overline{S} = \{u \mid \vec{d}_u \geq k\}$, so the invariant holds. A call to REACHOUT searches for a reversible hyperpath from

$V \setminus \overline{S}$ into $\overline{S}$. Reversing such a path increases the outside endpoint by one and decreases the inside endpoint by one; if the outside endpoint reaches indegree k, it is inserted into $\overline{S}$. Hence every discovered external vertex that can be promoted is promoted, and any inside vertex whose indegree drops below k is subsequently handled by OUT. A call to REACHIN is symmetric: it removes reversible reachability from a deficient candidate to outside vertices before the candidate is discarded. Finally, OUT exhausts reversible hyperpaths involving the deficient vertex and then removes it only if its indegree remains below k. When the while-loop terminates, no vertex in $\overline{S}$ has indegree below k, and the preceding repair steps ensure that no reversible hyperpath can move contribution across the boundary in a way that would either promote an outside vertex into $\overline{S}$ or restore a removed vertex. Thus $\overline{S}$ is exactly the stable seed set $\{u \mid \vec{d}_u \geq k\}$ in the threshold-stable orientation obtained by the algorithm. The returned set is $\overline{S}$ plus all vertices that can reach $\overline{S}$, which matches Definition 4. Uniqueness from Theorem 2 then implies that the output is the unique (k, δ)-dense subhypergraph $D_{k,\delta}$.

Theorem 11 (Complexity of Algorithm DSM-ALL*):* The time and space complexity are $O(nm\delta)$ and $O(n + m)$.

Proof. A search for a reversible hyperpath can be implemented by BFS on the incidence representation of the oriented hypergraph. Each directed hyperedge has at most δ target incidences and at most $|e|$ source incidences; scanning all possible source–target transitions over the whole hypergraph costs $O(m\delta)$ under the bounded-target representation used by the algorithm. The reversal of a found hyperpath is charged to the search that found it and does not asymptotically increase this cost. Each vertex can become deficient and invoke OUT only after its indegree has been decreased by a previous reversal, and it is removed from $\overline{S}$ once it cannot be repaired. Similarly, each vertex outside $\overline{S}$ triggers REACHOUT only while it is being tested for promotion. Therefore there are $O(n)$ such vertex-level searches in the worst case. Multiplying the $O(n)$ searches by the $O(m\delta)$ cost of each search gives $O(nm\delta)$ time. The algorithm stores the oriented incidence lists, the current indegrees, the membership flag of $\overline{S}$, and BFS marks/queues. These structures require $O(n+m)$ space when hyperedge incidences are represented in the input adjacency lists, so the space complexity is $O(n + m)$.

IV. DENSITY DECOMPOSITION

To efficiently solve the decomposition problem introduced in Section 2, we develop two algorithms for computing all non-empty (k, δ)-dense subhypergraphs: a basic iterative version, DSD, and an enhanced divide-and-conquer variant, DSD+. The baseline DSD algorithm incrementally extracts $D_{1,\delta}, D_{2,\delta}, \ldots, D_{k_{\max},\delta}$ by repeatedly invoking a subroutine that identifies a single (k, δ)-dense subhypergraph. However, this iterative process introduces redundancy, as many intermediate results are recomputed multiple times. To overcome this inefficiency, DSD+ leverages the hierarchical structure of dense subhypergraphs to decompose the problem recursively and reuse partial results across subproblems, thus substantially improving efficiency without compromising completeness.

Algorithm 5: DSD($\mathcal{H}$)

```
Input: a hypergraph H = (V, E).
Output: all non-empty D_{k,δ} of H
1 for δ = 1, 2, ... do
2     Arbitrarily obtain an δ-orientation H⃗ of H;
3     for k = 1, 2, 3, ... do
4         DSM-ALL (H⃗, k, δ);
5         if D_{k,δ} = ∅ then break;
6     return D = {D_{k,δ}};
```

A. The Basic Decomposition Algorithm: DSD

Using DSM-ALL as a subroutine, the density subhypergraph decomposition can be computed layer by layer. Based on this idea, we propose the DSD algorithm, as shown in Algorithm 5. DSD enumerates different density levels by iterating over all feasible pairs of δ and k (Lines 1–3). For each fixed pair, it invokes DSM-ALL$(\vec{\mathcal{H}}, k, \delta)$ to extract the corresponding (k, δ)-dense subhypergraph (Line 4).

The correctness of DSD follows directly from DSM-ALL and Theorems 2 and 3, so we omit the proof.

Theorem 12 (Complexity of Algorithm DSD*):* The time and space complexity are $O(nm\delta \cdot d^E_{max} \cdot k_{max})$ and $O(n + m)$.

Proof. For each value of δ, DSD first constructs a δ-orientation in $O(m\delta)$ time and then performs peeling over at most $k_{\max}$ layers. In each layer, up to $O(n)$ vertices may trigger OUT, and each call costs $O(m\delta)$ in the worst case. Since δ ranges up to $d^E_{\max}$, the total time complexity is $O(nm\delta \cdot d^E_{\max} \cdot k_{\max})$. The algorithm maintains only the oriented hypergraph and auxiliary vertex states, so the space complexity is $O(n + m)$.

B. The Improved Decomposition Algorithm: DSD+

DSD computes the decomposition in a layer-by-layer manner, sequentially deriving $D_{1,\delta}, D_{2,\delta}, \ldots, D_{k_{\max},\delta}$ for each fixed δ. However, these layers are highly nested, since $D_{k+1,\delta} \subseteq D_{k,\delta}$. As a result, many vertices and hyperedges that have already been processed at lower density levels may be revisited when computing higher layers. To reduce this redundancy, we propose DSD+, a divide-and-conquer variant that reuses intermediate results across density levels.

As shown in Algorithm 6, for each δ, DSD+ first determines the maximum non-empty layer $k_{\max}$ by binary search (Line 3). It then invokes DIVIDE$(D_{1,\delta}, D_{k_{\max},\delta})$ to recover all non-empty layers recursively (Line 4). Within DIVIDE, the algorithm first checks whether the current interval has been fully resolved (Line 7). If not, it selects the midpoint k_m and computes the intermediate layers around the split point using DSM-ALL (Lines 9–10). The algorithm then recursively processes only the corresponding subhypergraph differences rather than the entire graph. By exploiting the nesting property $D_{k+1,\delta} \subseteq D_{k,\delta}$, DSD+ avoids revisiting regions that have already been assigned to a density layer.

Algorithm 6: DSD+($\mathcal{H}$)

Input: a hypergraph $\mathcal{H} = (V, E)$.
Output: all non-empty $D_{k,\delta}$ of $\mathcal{H}$

for $\delta = 1, 2, 3, \ldots$ **do**
  Arbitrarily obtain an δ-orientation $\vec{\mathcal{H}}$ of $\mathcal{H}$, $D_{1,\delta} \leftarrow V$;
  $k_{max} \leftarrow$ the maximum integral such that $D_{k_{max},\delta} \neq \emptyset$;
  DIVIDE $(D_{k_{max},\delta}, D_{1,\delta})$;
return $\mathcal{D} = \{D_{k,\delta}\}$;
Function DIVIDE $(D_{k_u,\delta}, D_{k_l,\delta})$ **:**
  if $k_u - k_l \leq 1$ *or* $D_{k_u,\delta} = D_{k_l,\delta}$ **then**
    return;
  $k_m \leftarrow (k_u + k_l + 1)/2$, DSM-ALL $(\vec{\mathcal{H}}, k_m, \delta)$;
  DIVIDE $(D_{k_u,\delta}, D_{k_m,\delta})$, DIVIDE $(D_{k_m,\delta}, D_{k_l,\delta})$;

Theorem 13 (Correctness of Algorithm DSD+*):* Given a hypergraph $\mathcal{H} = (V, E)$, Algorithm DSD+ correctly computes all non-empty (k, δ)-dense subhypergraphs $D_{k,\delta}$, and each such subhypergraph is computed exactly once.

Proof. Fix δ. By Theorem 3, the family $D_{1,\delta}, D_{2,\delta}, \ldots, D_{k_{\max},\delta}$ is nested. Therefore, for any two already computed boundary layers $D_{k_l,\delta}$ and $D_{k_u,\delta}$ with $k_l < k_u$, every unresolved layer whose index lies in (k_l, k_u) must be contained in $D_{k_l,\delta}$ and must contain $D_{k_u,\delta}$. No layer outside this interval can appear inside the recursive subproblem. Algorithm 6 computes the midpoint layer $D_{k_m,\delta}$ using the correct single-layer procedure DSMext−ALL. By Theorem **??**, the computed midpoint is the unique correct (k_m, δ)-dense subhypergraph. The midpoint splits the unresolved interval into two independent intervals, (k_l, k_m) and (k_m, k_u), and the nesting property guarantees that recursively processing these two intervals loses no feasible layer and introduces no layer with an index outside the interval. The recursion stops only when adjacent indices remain or when the two boundary vertex sets are equal. In the first case there is no integer level between the boundaries. In the second case, nesting implies that every intermediate layer is equal to the same vertex set, so no distinct non-empty subhypergraph remains to be discovered. Since each recursive call uses a previously unseen midpoint index, each distinct non-empty layer is computed at most once, and the above completeness argument shows that all of them are computed. Repeating this argument for every δ gives the full decomposition.

Theorem 14 (Complexity Analysis): The time and space complexity are $O(nm\delta \cdot d^E_{max} \log k_{max})$ and $O(n + m)$.

Proof. DSD+ recursively bisects the density interval and invokes DSM−ALL on intermediate layers. As the recursion depth is $O(\log k_{\max})$ and each DSM−ALL call runs in $O(nm\delta)$ time, the total complexity is $O(nm\delta \cdot d^E_{\max} \log k_{\max})$. Space usage remains linear $O(n+m)$ since each recursive call operates in-place without duplicating the hypergraph structure.

Example 4: Figure 4 illustrates the DIVIDE process on the HB dataset with fixed $\delta = 5$. The initial call DIVIDE$(D_{1,5}, D_{316,5})$ covers 1493 vertices and selects $k = 159$ as the midpoint. The recursion then splits the range into $[1, 159]$ and $[159, 316]$, with corresponding vertex sets of sizes 558 and 935. Subsequent calls continue this process by selecting new midpoints, such as $k = 80$ and $k = 238$, until a base case is reached, either when the interval length is at most one or when adjacent layers contain the same vertex set.

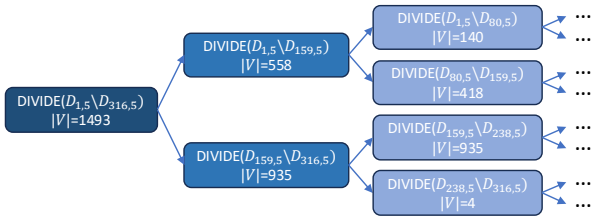


Fig. 4. An example of DIVIDE $(D_{1,5}, D_{316,5})$ on HB dataset.

This recursion tree demonstrates how DSD+ reduces redundant computation by reusing intermediate layers and restricting later calls to smaller subhypergraph differences. In Figure 4, lighter-colored blocks correspond to smaller vertex sets, showing that the workload gradually decreases at deeper recursion levels. Thus, the strategy efficiently recovers all distinct non-empty (k, δ)-dense subhypergraphs with low overhead.

V. Dynamic Algorithms

Real-world hypergraphs often evolve over time as new relations appear and old ones vanish. To handle such updates efficiently, we design a dynamic maintenance mechanism that incrementally updates the egalitarian orientation and IDNs without reconstructing the entire decomposition.

Edge addition (Algorithm 7) incrementally integrates a new hyperedge e into the current egalitarian orientation $\vec{\mathcal{H}}$. For each δ iteration (Lines 1–3), it extends the orientation by inserting e while maintaining the non-decreasing indegree order of its incident vertices. For every affected vertex $v \in e$ (Line 4), the algorithm checks whether its indegree $\vec{d}^{\delta}_v$ reaches the limit $\bar{r}^{\delta}_v$ (Line 5). If so, it searches for a *reversible hyperpath* $s \rightsquigarrow v$ (Line 6) whose reversal restores balance (Line 7); otherwise, it propagates local adjustments to reachable vertices sharing the same IDN with v (Lines 8–11). Finally, the updated indegrees $\bar{r}$ and orientation $\vec{\mathcal{H}}$ are returned (Line 12).

Edge deletion (Algorithm 8) symmetrically removes a hyperedge e from the current egalitarian orientation $\vec{\mathcal{H}}$ while maintaining egalitarian. For each δ iteration (Lines 1–3), the algorithm deletes e and inspects affected vertices $v \in e$ (Line 4). If $\vec{d}_v$ decreases by two below its IDN $\bar{r}^{\delta}_v$ (Line 5), a *reversible hyperpath* $v \rightsquigarrow t$ must exist such that the indegree difference between v and t equals 2, and this hyperpath is then reversed to restore balance (Lines 6–7). Afterward, all vertices with the same IDN $\bar{r}^{\delta}_v$ that can reach either v or t are collected into a temporary set P, including v and t (Line 8). Otherwise, P contains vertices with the same IDN that can reach v, including v itself(Lines 9–10). For each $w \in P$, if $\vec{d}_w$ is lower than its IDN and w cannot reach any vertex with

Algorithm 7: Insert($\overrightarrow{\mathcal{H}}, \bar{r}, e$)

Input: The egalitarian orientation $\overrightarrow{\mathcal{H}}$, the IDNs of all vertices $\bar{r}$, and the hyperedge e to be inserted.

Output: The updated egalitarian orientation $\overrightarrow{\mathcal{H}}$ and IDNs $\bar{r}$

1 **for** $\delta = 1, 2, ...$ **do**
2 Suppose $e = (u_1, u_2, ..., u_i, ...)$, $\vec{d}_{u_i} \leq \vec{d}_{u_{i+1}}$;
3 $\vec{e} = (\{u_{\delta+1}, u_{\delta+1}, ...\}\{u_1, u_2, ..., u_\delta\})$, $\overrightarrow{\mathcal{H}} \leftarrow \overrightarrow{\mathcal{H}} \cup \vec{e}$;
4 **for** *each* $v \in \{u_1, u_2, ..., u_\delta\}$ **do**
5 **if** $\vec{d}_v = \bar{r}_v^\delta$*+1* **then**
6 **if** $\exists$ *reversible hyperpath* $s \rightsquigarrow v$*, with* $\vec{d}_s = \bar{r}_v^\delta - 1$ **then**
7 reverse the hyperpath $s \rightsquigarrow v$;
8 **else**
9 **for** *each* $w \in \{w | \bar{r}_w^\delta = \bar{r}_v^\delta\}$*and* w *can reach* v **do**
10 $\bar{r}_w^\delta \leftarrow \bar{r}_w^\delta + 1$;
11 $\bar{r}_v^\delta \leftarrow \bar{r}_v^\delta + 1$;
12 **return** $(\overrightarrow{\mathcal{H}}, \bar{r})$;

Algorithm 8: Delete($\overrightarrow{\mathcal{H}}, \bar{r}, e$)

Input: The egalitarian orientation $\overrightarrow{\mathcal{H}}$, the IDNs of all vertices $\bar{r}$, and the hyperedge e to be deleted.

Output: The updated egalitarian orientation $\overrightarrow{\mathcal{H}}$ and IDNs $\bar{r}$

1 **for** $\delta = 1, 2, 3, ...$ **do**
2 Suppose e is oriented as $\vec{e} = (\{u_{\delta+1}, u_{\delta+1}, ...\}\{u_1, u_2, ..., u_\delta\})$;
3 $\overrightarrow{\mathcal{H}} \leftarrow \overrightarrow{\mathcal{H}} \setminus \vec{e}$;
4 **for** *each* $v \in \{u_1, u_2, ..., u_\delta\}$ **do**
5 **if** $\vec{d}_v = \bar{r}_v^\delta - 2$ **then**
6 must $\exists$ a reversible hyperpath $v \rightsquigarrow t$, with $d_t = \bar{r}_v^\delta$;
7 reverse the hyperpath $v \rightsquigarrow t$;
8 $P \leftarrow \{w | \bar{r}_w^\delta = \bar{r}_v^\delta$, and w can each v or $t\} \cup v \cup t$;
9 **else**
10 $P \leftarrow \{w | \bar{r}_w^\delta = \bar{r}_v^\delta$, and w can each $v\} \cup v$;
11 **for** *each* $w \in P$ **do**
12 **if** $d_w \neq \bar{r}_w^\delta$ *and can't reach an* $\bar{r}_w^\delta$*-indegree vertex* **then**
13 $\bar{r}_w^\delta \leftarrow \bar{r}_w^\delta - 1$;
14 **return** $(\overrightarrow{\mathcal{H}}, \bar{r})$;

indegree $\bar{r}_w^\delta$, its IDN is decreased (Lines 11–13). Finally, the updated orientation and IDNs are returned (Line 14).

Together, the insertion and deletion procedures maintain the decomposition through localized hyperpath reversals and IDN updates. By restricting the repair process to affected vertices and their reachable regions, they preserve the egalitarian orientation without recomputing the decomposition from scratch.

TABLE I
DATA HYPERGRAPH $\mathcal{H}$.

Datasets	$\lvert V\rvert = n$	$\lvert E\rvert = m$	d^E_{max}	d^E_{min}	m/n	$\bar{d_e}$
CP	242	12,704	5	2	52.5	2.42
SC	282	315	31	4	1.12	17.6
SB	294	29,157	99	2	99.2	9.65
CH	327	7,818	5	2	23.9	2.33
HC	1,290	341	82	1	0.26	35.2
HB	1,494	60,987	399	2	40.8	21.9
TC	172,738	233,202	85	2	1.35	3.18
AR	2,268,231	4,285,363	9,350	2	1.88	17.1
SA	15,211,989	1,103,243	61,315	2	0.07	23.7

VI. EXPERIMENTAL EVALUATION

A. Experimental setup

In this section, we evaluate the performance of the algorithms we proposed across various datasets. All algorithms are implemented with C++ and compiled using gcc version 11.1.0 with optimization level set to O3. All experiments are conducted on a Linux machine equipped with a 2.9GHz AMD Ryzen 3990X CPU and 256GB RAM running CentOS 7.9.2 (64-bit). The experimental results are meticulously detailed in the subsequent parts of this section.

Datasets. We evaluate our methods on nine benchmark hypergraph datasets widely used in prior decomposition studies. Table I summarizes their key statistics. All datasets are publicly available at https://www.cs.cornell.edu/~arb/data/ and are listed in ascending order of vertex count. Specifically, CP (*contact-primary-school*) and CH (*contact-high-school*) record proximity interactions among students; SC (*senate-committee*) and HC (*house-committees*) capture committee memberships in the US Senate and House of Representatives; SB (*senate-bills*) and HB (*house-bills*) represent bill co-sponsorship networks in the US Congress; TC (*trivago-clicks*) logs hotels co-clicked during online browsing sessions; AR (*amazon-reviews*) groups products reviewed by individual users on Amazon; and SA (*stackoverflow-answers*) aggregates questions answered by the same user on Stack Overflow.

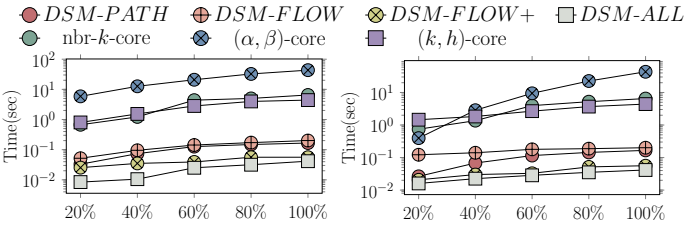
Algorithms. We implement six algorithms: four (k, δ)-dense subhypergraph mining algorithms, namely DSM-PATH, DSM-FLOW, DSM-FLOW+, and DSM-ALL (Algorithms 1–4), and two density decomposition algorithms, DSD and DSD+ (Algorithms 5–6). Since (k, δ)-dense subhypergraph is a new model, there are no existing algorithms that directly compute it or its decomposition. For comparison, we also include several representative hypergraph decomposition algorithms: k-core [4], E-Peel [5] (also referred to as nbr-k-core), (α, β)-core [10], CoCoreDecomp [11] (i.e., (k, h)-core), densest [12] and HTC-PF (also referred to as hyper k-truss) [3]. To facilitate comparison, we align the parameters across models by mapping k (or α) in these methods to the k in our model, and mapping h, β to our δ.

B. Efficiency Testings

Exp-1: Runtime of subhypergraph mining algorithms. Table II summarizes the runtime of different subhypergraph mining algorithms across multiple datasets. In the table,

TABLE II
RUNTIME OF SUBHYPERGRAPH MINING ALGORITHMS (SEC).

Methods	CP	SB	CH	HC	HB	TC	AR	SA
DSM-PATH	0.007	0.035	0.007	0.002	0.173	273.4	50.07	4.992
DSM-FLOW	0.005	0.032	0.005	0.006	0.151	201.2	OOM	OOM
DSM-FLOW+	0.004	0.015	0.005	0.003	0.055	31.68	OOM	OOM
DSM-ALL	**0.002**	**0.009**	**0.004**	**0.002**	**0.042**	**0.164**	**42.17**	**4.609**
nbr-k-core	0.004	0.216	0.005	0.059	6.708	0.594	554.5	2409
(α, β)-core	0.132	14.87	0.031	0.002	43.33	0.236	345.2	UNM
(k, h)-core	0.005	0.377	0.006	0.015	6.828	0.278	170.2	32.34



(a) vary $|V|$ on HB (b) vary $|E|$ on HB

Fig. 5. Scalability of subhypergraph mining algorithms.

"OOM" indicates out-of-memory errors and "UNM" denotes cases exceeding 12 hours. On the HB dataset (k=5, δ=5), all our methods substantially outperform traditional baselines: DSM−ALL completes in 0.042 seconds, whereas nbr-k-core requires 6.708 seconds—over 160× slower. This pattern holds consistently across datasets, where our algorithms finish within seconds and deliver significant speedups over classical models. Compared with DSM−PATH and DSM−FLOW, which incur additional cost due to fine-grained hyperpath reversals or flow construction, DSM−ALL is the most stable and practical choice on large hypergraphs. A more fine-grained study of the impact of δ is deferred to the ablation experiment in Exp-9.

Exp-2: Scalability of subhypergraph mining algorithms. Figure 5 evaluates the scalability of subhypergraph mining algorithms on the HB dataset by varying $|V|$ and $|E|$. In both settings, DSM−ALL achieves the best scalability, maintaining sub-second runtimes with minimal growth as the hypergraph expands, showing strong resilience to both vertex and hyperedge increases. DSM−FLOW+ and DSM−PATH also scale well, with slight overhead from finer-grained operations. In contrast, classical models like (α, β)-core and (k, h)-core show steep runtime growth, underscoring the superior efficiency and robustness of our methods—especially DSM−ALL.

Exp-3: Runtime of decomposition algorithms. Figure 6 compares the runtime of seven decomposition algorithms across multiple hypergraph datasets. Our methods, DSD and DSD+, consistently achieve lower runtimes, showing strong efficiency and scalability. In particular, DSD+ attains notable speedups by reusing intermediate results via divide-and-conquer. On TC, nbr-k-core performs slightly faster due to its local peeling design, while the densest model runs faster than DSD on HC as it extracts only a single maximum-density subhypergraph. The k-truss method is also competitive on HC and TC thanks to parallelization (64 threads). In contrast, (α, β)-core and (k, h)-core incur heavy overhead from global computations and complex auxiliary structures.

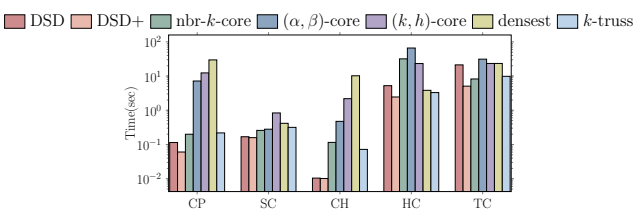
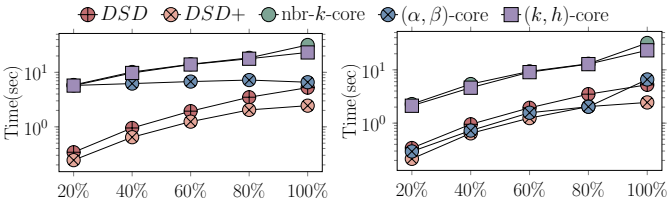


Fig. 6. Runtime of decomposition algorithms.



(a) vary $|V|$ on HC (b) vary $|E|$ on HC

Fig. 7. Scalability of decomposition algorithms.

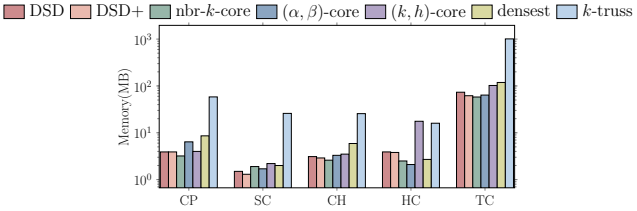


Fig. 8. Memory overheads of decomposition algorithms.

Exp-4: Scalability. Figure 7 evaluates the scalability of decomposition algorithms on the HC dataset by varying $|V|$ and $|E|$. As $|V|$ increases, DSD and DSD+ show the most stable growth, indicating their scalability on large graphs and steady behavior as the vertices expands. In contrast, baselines such as (k, h)-core and nbr-k-core slow down sharply as the vertices grows, especially when the number of vertices becomes large. When varying $|E|$, DSD+ consistently achieves the lowest runtime across different densities, demonstrating robust performance under increasing numbers of hyperedges and higher decomposition workloads. Overall, these results confirm the excellent scalability and efficiency of our framework.

Exp-5: Memory overheads of decomposition algorithms. Figure 8 compares memory overheads of different decomposition algorithms across real-world hypergraphs. Our methods, DSD and DSD+, maintain stable and low memory usage, reflecting the lightweight design that avoids storing deep hierarchies or redundant metadata. In contrast, core-based methods like (k, h)-core and (α, β)-core consume more and less stable memory, especially on HC and TC. The *densest* baseline adds modest overhead for global density tracking, while k-truss shows the highest memory cost due to motif counting and multi-threaded edge-support maintenance.

C. Effectiveness Testings

Exp-6: Comparisons of the total hierarchy layers. Figure 9 compares the total number of hierarchy layers generated by

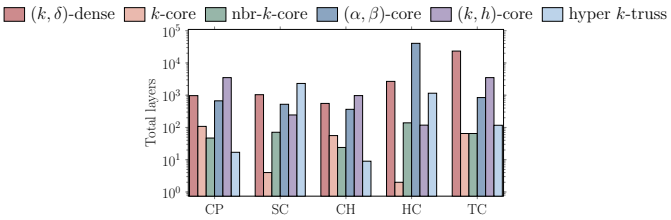
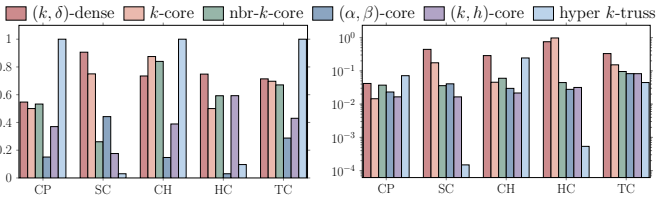


Fig. 9. Comparisons of the total hierarchy layers.



(a) Non-empty Layer Ratio (b) Average Layer Jaccard Distance

Fig. 10. Layer quality comparisons of the subhypergraph.

different decomposition models on five datasets. Our (k,δ)-dense decomposition produces consistently rich hierarchies, achieving the largest layer count on TC and showing competitive depth on SC. Single-parameter methods, including k-core, nbr-k-core, and hyper k-truss, generally produce fewer layers, suggesting a coarser decomposition granularity. An exception appears on SC, where hyper k-truss yields the largest number of layers, likely due to the presence of relatively large hyperedges that enable more fine-grained truss peeling. Dual-parameter methods, such as (k,h)-core and (α,β)-core, also produce deeper hierarchies on some datasets, reflecting their ability to distinguish structures along two constraints. Overall, the results indicate that the (k,δ)-dense model provides a fine-grained and stable hierarchical view across datasets, while avoiding reliance on a single density criterion.

Exp-7: Layer quality comparison. Figure 10 evaluates decomposition quality using two metrics: the non-empty layer ratio, which measures hierarchy continuity, and the average Jaccard distance between adjacent layers, which measures layer distinctness. The (k,δ)-dense decomposition consistently achieves a high non-empty layer ratio, indicating that its hierarchy contains many valid and continuous layers. Although (α,β)-core and (k,h)-core may generate many layers, a smaller fraction of them are non-empty, suggesting weaker continuity across the hierarchy. On CH, k-core and nbr-k-core obtain slightly higher ratios, likely due to the denser vertex connectivity in this dataset. For layer distinctness, Figure 10(b) shows that (k,δ)-dense decomposition ranks among the best methods in most cases, indicating clear transitions between adjacent layers. Hyper k-truss also achieves strong continuity on CP, CH, and TC, where motif-based peeling can effectively capture cohesive local structures. Overall, (k,δ)-dense decomposition provides a favorable balance between continuity and distinctness, producing stable and informative hierarchies across different hypergraphs.

Exp-8: Maximum density of subhypergraph models. We evaluate each subhypergraph model by its ability to capture the densest structures under three complementary metrics: (i) edge–vertex ratio ($|E|/|V|$), (ii) degree density (Definition 8), and (iii) volume density [5], which measures the average neighborhood size within the induced subhypergraph. Table III summarizes the maximum density achieved by each model across datasets. For decomposition-based methods (k-core, nbr-k-core, hyper k-truss, and (k,δ)-dense), we report the layer with the highest density, while the *densest* baseline [12] is adapted to each metric for fair comparison. Overall, (k,δ)-dense consistently attains the highest or near-highest values, confirming its strength in preserving cohesive and fine-grained dense structures within a unified hierarchy.

TABLE III
MAXIMUM DENSITY OF SUBHYPERGRAPH MODELS.

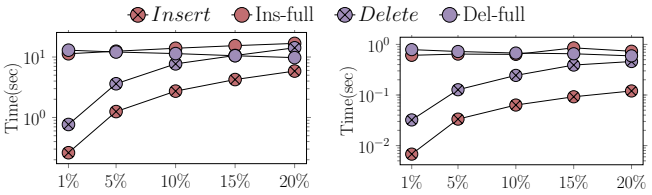
Density Metrics	Methods	CP	SC	CH	HC	HB	TC
|E|/|V|	k-core	54.47	**1.128**	25.58	0.750	**38.20**	17.78
	nbr-k-core	53.67	**1.128**	25.40	0.260	37.90	2.804
	hyper k-truss	50.40	1.124	21.62	0.257	OOM	1.510
	densest	**54.48**	**1.128**	**25.60**	**1**	**38.20**	UNM
	(k,δ)-dense	**54.48**	**1.128**	**25.60**	0.692	**38.20**	**18.35**
volume-density	k-core	70.51	105.5	36.73	195.6	667.2	30.77
	nbr-k-core	**71.01**	108.2	**36.89**	**216.4**	**676.8**	**88.63**
	k-truss	67.31	105.4	33.20	197.0	OOM	44.98
	densest	**71.01**	**108.3**	**36.89**	**216.4**	UNM	UNM
	(k,δ)-dense	70.25	105.5	36.74	195.6	666.9	32.31

TABLE IV
EFFECT OF δ ON $k_{\max}$, SAT, AND CONT ACROSS DATASETS.

Datasets	Metrics	1	$\lfloor\sqrt{\bar{d}_e}\rfloor$	d_{50}	$\lfloor\bar{d}_e\rfloor$	d_{75}	d_{95}	d^E_{max}
SB	k_{max}	84	279	734	1138	1374	2695	3264
	Sat	1	0.689	0.442	0.308	0.249	0.048	0
	Cont	0.996	0.988	0.993	0.995	0.996	0.998	0.998
HB	k_{max}	40	243	804	1790	2126	4636	6179
	Sat	1	0.712	0.477	0.291	0.243	0.049	0
	Cont	0.981	0.979	0.992	0.996	0.997	0.998	0.999
TC	k_{max}	19	19	101	179	179	262	284
	Sat	1	1	0.474	0.248	0.248	0.040	0
	Cont	0.662	0.662	0.897	0.941	0.941	0.958	0.961

Exp-9: Ablation on δ and practical guidance. To investigate how the parameter δ affects decomposition, we evaluate three metrics: the number of layers ($k_{\max}$), hyperedge saturation ratio (Sat), and inter-layer continuity (Cont). Sat measures the fraction of truncated hyperedges, while Cont quantifies smoothness between consecutive layers via average Jaccard similarity. As shown in Table IV, δ governs the trade-off between coverage and compactness. When $\delta = 1$, nearly all edges are truncated (Sat $\approx$ 1), yielding coarse structures. As δ increases, Sat decreases and Cont approaches 1.0, indicating smoother, more interpretable hierarchies. Across datasets, moderate δ values around $\lfloor\bar{d}_e\rfloor$ or d_{75} achieve the best balance (Sat $\approx$ 0.2–0.4, Cont>0.9), suggesting a practical rule: set δ such that the saturation ratio falls within 20–40% for stable, well-layered decompositions without excessive computation.

Exp-10: Dynamic maintenance. We simulate dynamic hypergraphs by generating update batches that contain 1–20% of the original hyperedges, covering both insertions and deletions. Figure 11 compares incremental maintenance with full recom-



(a) Results on SB (b) Results on TC

Fig. 11. Efficiency of dynamic maintenance.

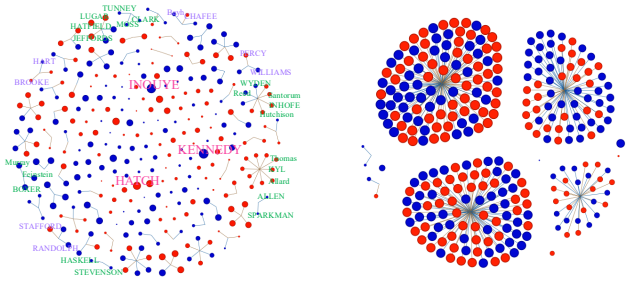
TABLE V
TOP-50 FRAUD SCREENING ON AMLSIM.

Methods	precision	recall	F1	ROC_AUC	PR_AUC
$(k,5)$-dense	**0.860**	**0.026**	**0.050**	**0.519**	**0.191**
k-core	0.800	0.020	0.042	0.519	0.189
nbr-k-core	0.516	0.020	0.030	0.563	0.170

putation on the SB and TC datasets. To avoid the prohibitive cost of evaluating all δ values, we fix $\delta = \lfloor \bar{d}_e \rfloor$, which has been shown to provide strong decomposition quality in Exp-8. For insertions, incremental maintenance consistently outperforms full recomputation across all update ratios, with runtime increasing almost linearly as the batch size grows. For deletions, it remains more efficient for small update batches ($\leq 10\%$), but its advantage gradually diminishes as the deletion ratio increases, approaching the cost of full recomputation at 20%. These results demonstrate that the proposed dynamic maintenance strategy is particularly effective for frequent small-scale updates, while full recomputation becomes competitive only when the update batch is sufficiently large.

D. Case studies

Case Study A: Detecting Fraud in Multi-party Financial Transactions. We evaluate our method on the AMLSim benchmark [13], [14], a public simulator that generates anti–money-laundering (AML) transaction graphs. Each alert represents a set of related transactions corresponding to a known fraud pattern, and all participating accounts are labeled as fraudulent. We model each alert as a hyperedge connecting all involved accounts, with additional background edges from normal transactions. This enables quantitative evaluation against ground-truth labels. We apply the $(k,5)$-dense decomposition ($\delta = 5 = \lfloor \bar{d}_e \rfloor$, validated in Exp-9) along with k-core and nbr-k-core baselines. Accounts from higher layers (larger k) are ranked and selected as top candidates under limited audit budgets. Detection quality is assessed using standard metrics: precision, recall, F1, ROC-AUC, and PR-AUC [14]. ROC-AUC measures ranking quality over all thresholds, while PR-AUC better reflects performance under severe class imbalance. As shown in Table V, the $(k,5)$-dense model attains the best precision, F1, and PR-AUC, outperforming k-core and nbr-k-core. Notably, precision improves by 7.5% over k-core under the same Top-50 audit budget, demonstrating that dense layers more effectively concentrate fraudulent accounts for real-world AML screening.



(a) $(k,10)$-dense (b) nbr-k-core

Fig. 12. Co-sponsorship community visualization.

Case Study B: Legislative Co-sponsorship Community Discovery. We evaluate our method on the U.S. Senate Bill dataset, where each bill forms a hyperedge linking all sponsors and legislators are labeled by party affiliation. Legislators (vertices) are labeled by party affiliation (red: Republicans, blue: Democrats). This dataset reflects real political collaborations and serves to assess both interpretability and predictive capability. Figure 12 compares our $(k,10)$-dense decomposition ($\delta = 10 = \lfloor \bar{d}_e \rfloor$) with k-core and nbr-k-core. The (k,δ)-dense layers reveal fine-grained communities that distinguish parties and highlight influential bipartisan legislators (*Hatch*, *Kennedy*, *Inouye*), within-party groups (*Murray*, *Feinstein*, *Boxer*), and cross-party collaborations. In contrast, baseline methods collapse these into coarse layers, obscuring structure.

VII. RELATED WORK

A. Graph Decomposition

Graph decomposition has been widely studied for discovering cohesive and dense structures. Classical cohesive models include k-core [15], [16], [17], k-truss [18], [19], [20], nucleus decomposition [21], [22], and k-edge-connected components [23], [24]. Other variants incorporate distance or attribute constraints, such as distance-generalized cores [25], [26] and colorful h-star k-core decompositions [27], [28]. Density-oriented studies include densest subgraph algorithms [29], [30], [31], locally densest subgraphs [32], [33], higher-order dense subgraph models [34], density-friendly decomposition [35], and bipartite density decomposition [36]. These methods motivate hierarchical density analysis, but they are defined on pairwise graphs or graph motifs rather than hyperedges as first-class high-order relations.

B. Hypergraph Decomposition

Cohesive and density-based decomposition. Hypergraph decomposition extends cohesive analysis from pairwise edges to group interactions. Core-style models include degree-based hypergraph k-core [37], [38], nbr-k-core [5], [7], (k,h)-core [11], (k,g)-core [39], (k,q)-core [40], [41], and (k,t)-core [42], which rely on degree, neighborhood, co-occurrence, hyperedge-size, or participation constraints. Density- and motif-based models, such as densest subhypergraph discovery [12], [43] and hypergraph truss decomposition [3], capture other forms of higher-order cohesiveness.

Structural hypergraph decomposition. Structural hypergraph decompositions focus on acyclicity, constraint satisfaction, and query tractability. Representative frameworks include k-hinge tree decomposition [44], hypertree decomposition [45], generalized hypertree decomposition [46], fractional hypertree decomposition [47], [48], and submodular-width based decomposition [49]. These methods target structural tractability, whereas our work focuses on density-aware hierarchical mining in hypergraphs.

VIII. Conclusion

In this paper, we studied density decomposition in hypergraphs. Existing approaches often struggle to accurately capture dense subhypergraphs while providing flexible control over hierarchical granularity. To address these limitations, we proposed the (k, δ)-dense subhypergraph model, which leverages hypergraph orientation to support adaptive and fine-grained density decomposition. The proposed model connects degree-based core decomposition with densest-subgraph analysis under a unified framework. We further developed four mining algorithms for computing (k, δ)-dense subhypergraphs, together with two decomposition algorithms for constructing the full density hierarchy. Extensive experiments on nine real-world hypergraphs demonstrate the effectiveness, efficiency, and scalability of the proposed framework.

REFERENCES

[1] M. Contisciani, F. Battiston, and C. D. Bacco, "Principled inference of hyperedges and overlapping communities in hypergraphs," *CoRR*, vol. abs/2204.05646, 2022.

[2] M. Mancastroppa, I. Iacopini, G. Petri, and A. Barrat, "Hyper-cores promote localization and efficient seeding in higher-order processes," *CoRR*, vol. abs/2301.04235, 2023.

[3] H. Qin, G. Zeng, R. Li, L. Lin, Y. Yuan, and G. Wang, "Truss decomposition in hypergraphs," *Proc. VLDB Endow.*, vol. 18, no. 7, pp. 2185–2197, 2025.

[4] Q. Luo, D. Yu, Z. Cai, X. Lin, and X. Cheng, "Hypercore maintenance in dynamic hypergraphs," in *ICDE*, 2021, pp. 2051–2056.

[5] N. A. Arafat, A. Khan, A. K. Rai, and B. Ghosh, "Neighborhood-based hypergraph core decomposition," *VLDB*, vol. 16, no. 9, pp. 2061–2074, 2023.

[6] H. Qin, R. Li, Y. Yuan, G. Wang, and Y. Dai, "Explainable hyperlink prediction: A hypergraph edit distance-based approach," in *ICDE*. IEEE, 2023, pp. 245–257.

[7] W. Zhang, Z. Yang, D. Wen, W. Li, W. Zhang, and X. Lin, "Accelerating core decomposition in billion-scale hypergraphs," *Proc. ACM Manag. Data*, vol. 3, no. 1, pp. 6:1–6:27, 2025.

[8] I. Bezáková, "Compact representations of graphs and adjacency testing," 2000.

[9] M. Blumenstock, "Fast algorithms for pseudoarboricity," in *ALENEX*. SIAM, 2016, pp. 113–126.

[10] D. Ding, H. Li, Z. Huang, and N. Mamoulis, "Efficient fault-tolerant group recommendation using alpha-beta-core," in *CIKM*, ser. CIKM '17. Association for Computing Machinery, 2017, p. 2047–2050.

[11] Q. Luo, D. Yu, Y. Liu, Y. Zheng, X. Cheng, and X. Lin, "Finer-grained engagement in hypergraphs," in *ICDE*, 2023, pp. 423–435.

[12] S. Hu, X. Wu, and T. H. Chan, "Maintaining densest subsets efficiently in evolving hypergraphs," in *CIKM 2017*, 2017, pp. 929–938.

[13] R. I. T. Jensen, J. Ferwerda, K. S. Jørgensen, E. R. Jensen, M. Borg, M. P. Krogh, J. B. Jensen, and A. Iosifidis, "A synthetic data set to benchmark anti-money laundering methods," *Scientific data*, vol. 10, no. 1, p. 661, 2023.

[14] E. R. Altman, J. Blanusa, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *NeurIPS*, 2023.

[15] V. Batagelj and M. Zaversnik, "An o(m) algorithm for cores decomposition of networks," *CoRR*, vol. cs.DS/0310049, 2003.

[16] R. Li, J. X. Yu, and R. Mao, "Efficient core maintenance in large dynamic graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 10, pp. 2453–2465, 2014.

[17] A. Montresor, F. D. Pellegrini, and D. Miorandi, "Distributed k-core decomposition," *IEEE Trans. Parallel Distributed Syst.*, vol. 24, no. 2, pp. 288–300, 2013.

[18] J. Cohen, "Trusses: Cohesive subgraphs for social network analysis," *National security agency technical report*, vol. 16, no. 3.1, pp. 1–29, 2008.

[19] J. Wang and J. Cheng, "Truss decomposition in massive networks," *VLDB*, vol. 5, no. 9, pp. 812–823, 2012.

[20] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying k-truss community in large and dynamic graphs," in *SIGMOD*. ACM, 2014, pp. 1311–1322.

[21] A. E. Sariyüce, C. Seshadhri, A. Pinar, and Ü. V. Çatalyürek, "Finding the hierarchy of dense subgraphs using nucleus decompositions," in *WWW*, 2015, pp. 927–937.

[22] J. Shi, L. Dhulipala, and J. Shun, "Theoretically and practically efficient parallel nucleus decomposition," *VLDB*, vol. 15, no. 3, pp. 583–596, 2021.

[23] L. Chang, J. X. Yu, L. Qin, X. Lin, C. Liu, and W. Liang, "Efficiently computing k-edge connected components via graph decomposition," in *SIGMOD*. ACM, 2013, pp. 205–216.

[24] L. Chang and Z. Wang, "A near-optimal approach to edge connectivity-based hierarchical graph decomposition," *VLDB J.*, vol. 33, no. 1, pp. 49–71, 2024.

[25] F. Bonchi, A. Khan, and L. Severini, "Distance-generalized core decomposition," in *SIGMOD*, 2019, pp. 1006–1023.

[26] Q. Dai, R. Li, L. Qin, G. Wang, W. Yang, Z. Zhang, and Y. Yuan, "Scaling up distance-generalized core decomposition," in *CIKM*, 2021, pp. 312–321.

[27] S. Gao, R. Li, H. Qin, H. Chen, Y. Yuan, and G. Wang, "Colorful h-star core decomposition," in *ICDE*, 2022, pp. 2588–2601.

[28] S. Gao, H. Qin, R. Li, and B. He, "Parallel colorful h-star core maintenance in dynamic graphs," *VLDB*, vol. 16, no. 10, pp. 2538–2550, 2023.

[29] A. V. Goldberg, "Finding a maximum density subgraph," 1984.

[30] M. Charikar, "Greedy approximation algorithms for finding dense components in a graph," in *International workshop on approximation algorithms for combinatorial optimization*. Springer, 2000, pp. 84–95.

[31] B. Bahmani, R. Kumar, and S. Vassilvitskii, "Densest subgraph in streaming and mapreduce," *arXiv preprint arXiv:1201.6567*, 2012.

[32] L. Qin, R. Li, L. Chang, and C. Zhang, "Locally densest subgraph discovery," in *SIGKDD*, 2015, pp. 965–974.

[33] N. Tatti, "Density-friendly graph decomposition," *ACM Trans. Knowl. Discov. Data*, vol. 13, no. 5, pp. 54:1–54:29, 2019.

[34] C. Tsourakakis, "The k-clique densest subgraph problem," in *Proceedings of the 24th international conference on world wide web*, 2015, pp. 1122–1132.

[35] Y. Zhang, R. Li, Q. Zhang, H. Qin, and G. Wang, "Efficient algorithms for density decomposition on large static and dynamic graphs," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 2933–2945, 2024.

[36] Y. Zhang, R. Li, Q. Zhang, H. Qin, L. Qin, and G. Wang, "Density decomposition of bipartite graphs," *Proc. ACM Manag. Data*, vol. 3, no. 1, pp. 30:1–30:25, 2025.

[37] E. Y. Ramadan, A. Tarafdar, and A. Pothen, "A hypergraph model for the yeast protein complex network," in *IPDPS*, 2004.

[38] G. Bianconi and S. N. Dorogovtsev, "Nature of hypergraph k-core percolation problems," *Physical Review E*, vol. 109, no. 1, p. 014307, 2024.

[39] D. Kim, J. Kim, S. Lim, and H. J. Jeong, "Exploring cohesive subgraphs in hypergraphs: The (k, g)-core approach," in *CIKM*, 2023, pp. 4013–4017.

[40] Q. Luo, W. Zhang, Z. Yang, D. Wen, X. Wang, D. Yu, and X. Lin, "Hierarchical structure construction on hypergraphs," in *CIKM*, 2024, pp. 1597–1606.

[41] J. Lee, K.-I. Goh, D.-S. Lee, and B. Kahng, "(k, q)-core decomposition of hypergraphs," *Chaos, Solitons & Fractals*, vol. 173, p. 113645, 2023.

[42] F. Bu, G. Lee, and K. Shin, "Hypercore decomposition for non-fragile hyperedges: concepts, algorithms, observations, and applications," *Data Min. Knowl. Discov.*, vol. 37, no. 6, pp. 2389–2437, 2023.

[43] Y. Huang, D. F. Gleich, and N. Veldt, "Densest subhypergraph: Negative supermodular functions and strongly localized methods," in *Proceedings of the ACM Web Conference 2024*, 2024, pp. 881–892.

[44] P. Jeavons, D. Cohen, and M. Gyssens, "A structural decomposition for hypergraphs," *Contemporary Mathematics*, vol. 178, pp. 161–161, 1994.

[45] G. Gottlob, N. Leone, and F. Scarcello, "Hypertree decompositions and tractable queries," *J. Comput. Syst. Sci.*, vol. 64, no. 3, pp. 579–627, 2002.

[46] D. A. Cohen, P. Jeavons, and M. Gyssens, "A unified theory of structural tractability for constraint satisfaction problems," *J. Comput. Syst. Sci.*, vol. 74, no. 5, pp. 721–743, 2008.

[47] M. Grohe and D. Marx, "Constraint solving via fractional edge covers," *CoRR*, vol. abs/1711.04506, 2017.

[48] G. Gottlob, M. Lanzinger, R. Pichler, and I. Razgon, "Complexity analysis of generalized and fractional hypertree decompositions," *Journal of the ACM (JACM)*, vol. 68, no. 5, pp. 1–50, 2021.

[49] D. Marx, "Tractable hypergraph properties for constraint satisfaction and conjunctive queries," *Journal of the ACM (JACM)*, vol. 60, no. 6, pp. 1–51, 2013.